important when trying to track down a bug that occurs “in the wild” as opposed to being detected in your test harness. Just because a bug occurs in the multithreaded portion of your application doesn’t mean it’s automatically concurrency-related. If you’re using thread pools to manage the level of concurrency, there’s usually a configuration parameter you can set to specify the number of worker threads. If you’re managing threads manually, you’ll have to modify the code to use a single thread for the test. Either way, if you can reduce your application to a single thread, you can eliminate concurrency as a cause. On the flip side, if the problem goes away on a *single-core* system (even with multiple threads running) but is present on *multicore* systems or *multiprocessor* systems, you have a race condition and possibly a synchronization or memory-ordering issue.

There’s more to testing concurrent code than the structure of the code being tested; the structure of the test is just as important, as is the test environment. If you continue on with the example of testing a concurrent queue, you have to think about various scenarios:

- One thread calling `push()` or `pop()` on its own to verify that the queue does work at a basic level
- One thread calling `push()` on an empty queue while another thread calls `pop()`
- Multiple threads calling `push()` on an empty queue
- Multiple threads calling `push()` on a full queue
- Multiple threads calling `pop()` on an empty queue
- Multiple threads calling `pop()` on a full queue
- Multiple threads calling `pop()` on a partially full queue with insufficient items for all threads
- Multiple threads calling `push()` while one thread calls `pop()` on an empty queue
- Multiple threads calling `push()` while one thread calls `pop()` on a full queue
- Multiple threads calling `push()` while multiple threads call `pop()` on an empty queue
- Multiple threads calling `push()` while multiple threads call `pop()` on a full queue